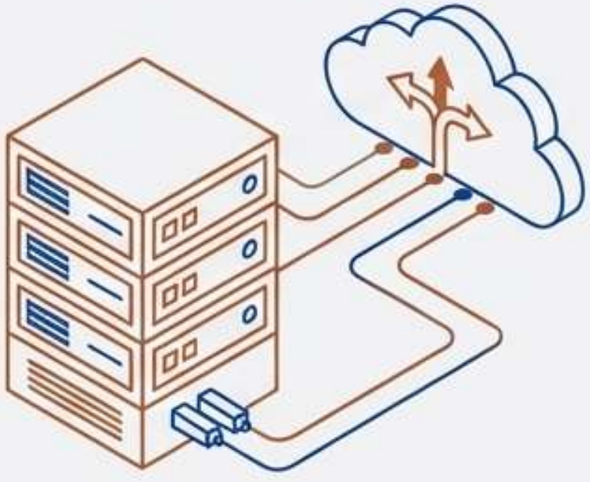
FSBM

FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Le Blueprint DevOps : De la Machine Locale au Cloud Automatisé

Déploiement d'une infrastructure CI/CD complète avec Kubernetes et AWS



Projet de **Master d'Excellence en Intelligence Artificielle (FSBM)**

Réalisé par : Rami Rajae & Bousnitra Douaa

Encadré par : Pr. F. Benabbou

Le changement de paradigme : Traditionnel vs. DevOps

Approche Traditionnelle

Organisation : Silos isolés entre les équipes Dev et Ops.

Déploiement : Manuel, lent et sujet aux erreurs humaines.

Infrastructure : Serveurs configurés manuellement (Fira Code: Snowflakes).

Résultat : Délais importants pour la mise en production.

Automatisation

Approche DevOps

Organisation : **Unification** et **collaboration continue**.

Déploiement : **Intégration** et **Déploiement Continus** (Fira Code: CI/CD).

Infrastructure : **Infrastructure as Code** (Fira Code: IaC), versionnée et reproductible.

Résultat : Livraison accélérée, haute **qualité** et **résilience**.

La chaîne d'outils DevOps : Carte de l'infrastructure

Provisioning

`Terraform`

Création automatisée des ressources cloud AWS.

Configuration

`Ansible`

Installation sans agent de Docker et Kubernetes.

Conteneurisation

`Docker`

Isolation de l'application Flask et de ses dépendances.

Orchestration

`Kubernetes`

Gestion, haute disponibilité et scalabilité des pods.

Automatisation

`GitHub Actions`

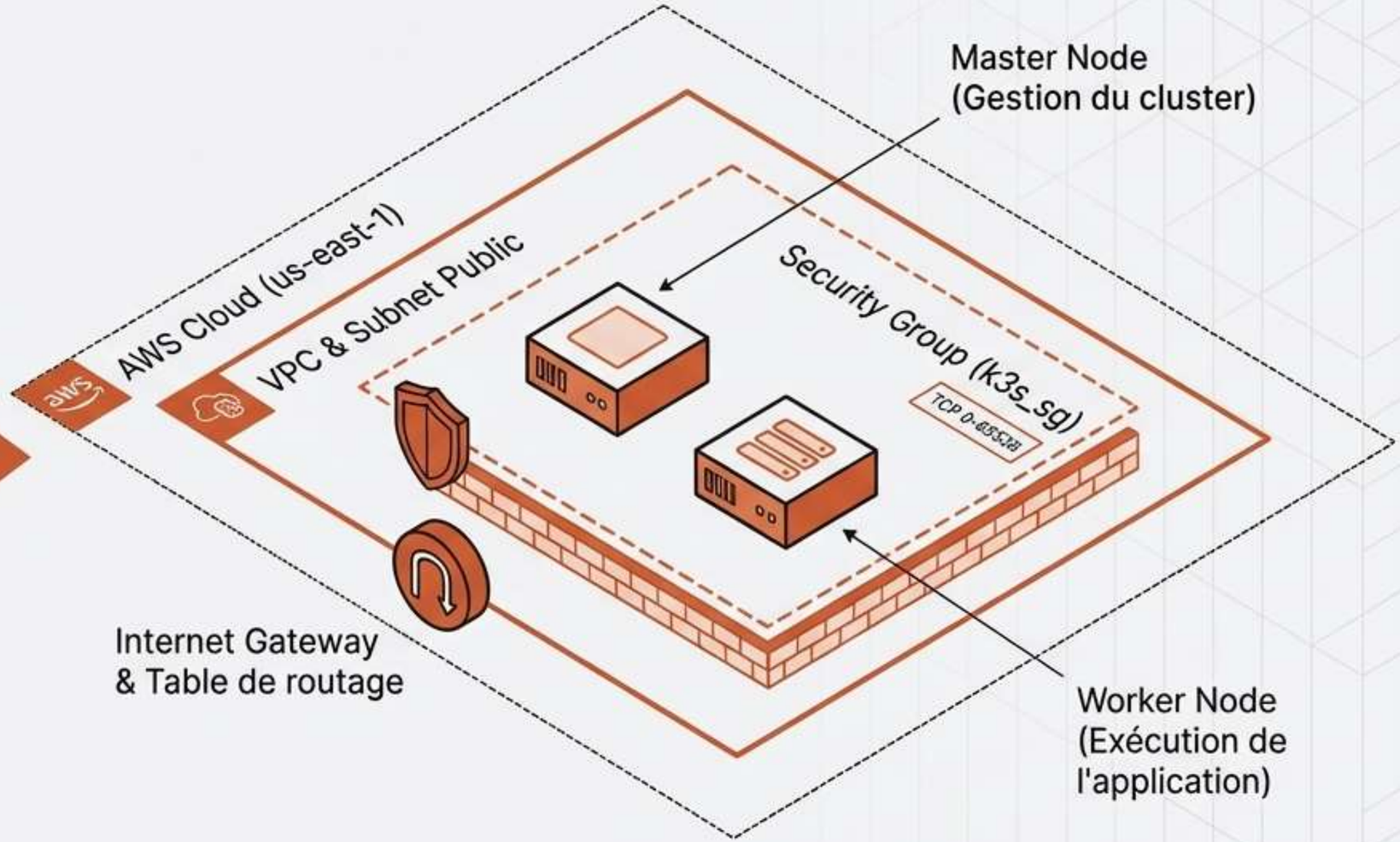
Moteur CI/CD orchestrant l'ensemble du flux.

Chaque étape est codifiée, versionnée et exécutée sans intervention manuelle.

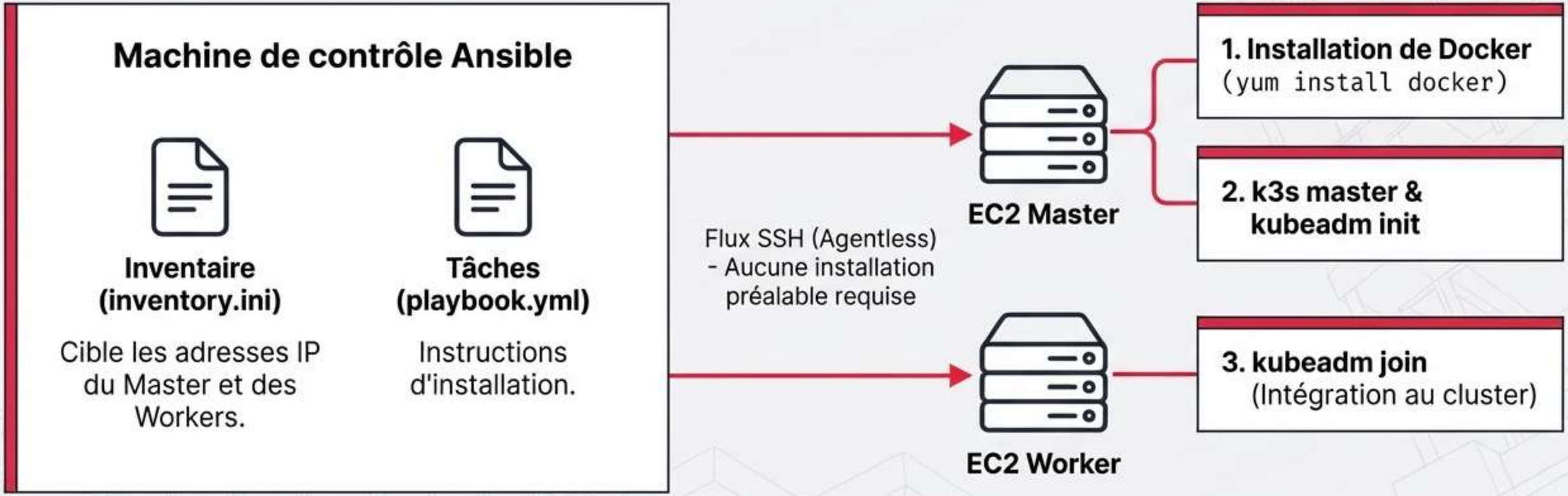
Les fondations cloud : **Provisionnement avec Terraform**

Terraform transforme 100% de cette infrastructure en code (main.tf, variables.tf), la rendant reproductible en une seule commande :

```
> terraform apply
```



La configuration sans agent : Déploiement avec Ansible



Le cœur du système : L'API Microservice (Flask)

Développement (app.py)

Route Principale /

```
{
  "message": "DevOps App DSBD",
  "status": "okk"
}
```

Délivre la valeur métier à l'utilisateur.

Route de Santé /health

```
{"status": "ok"}
```

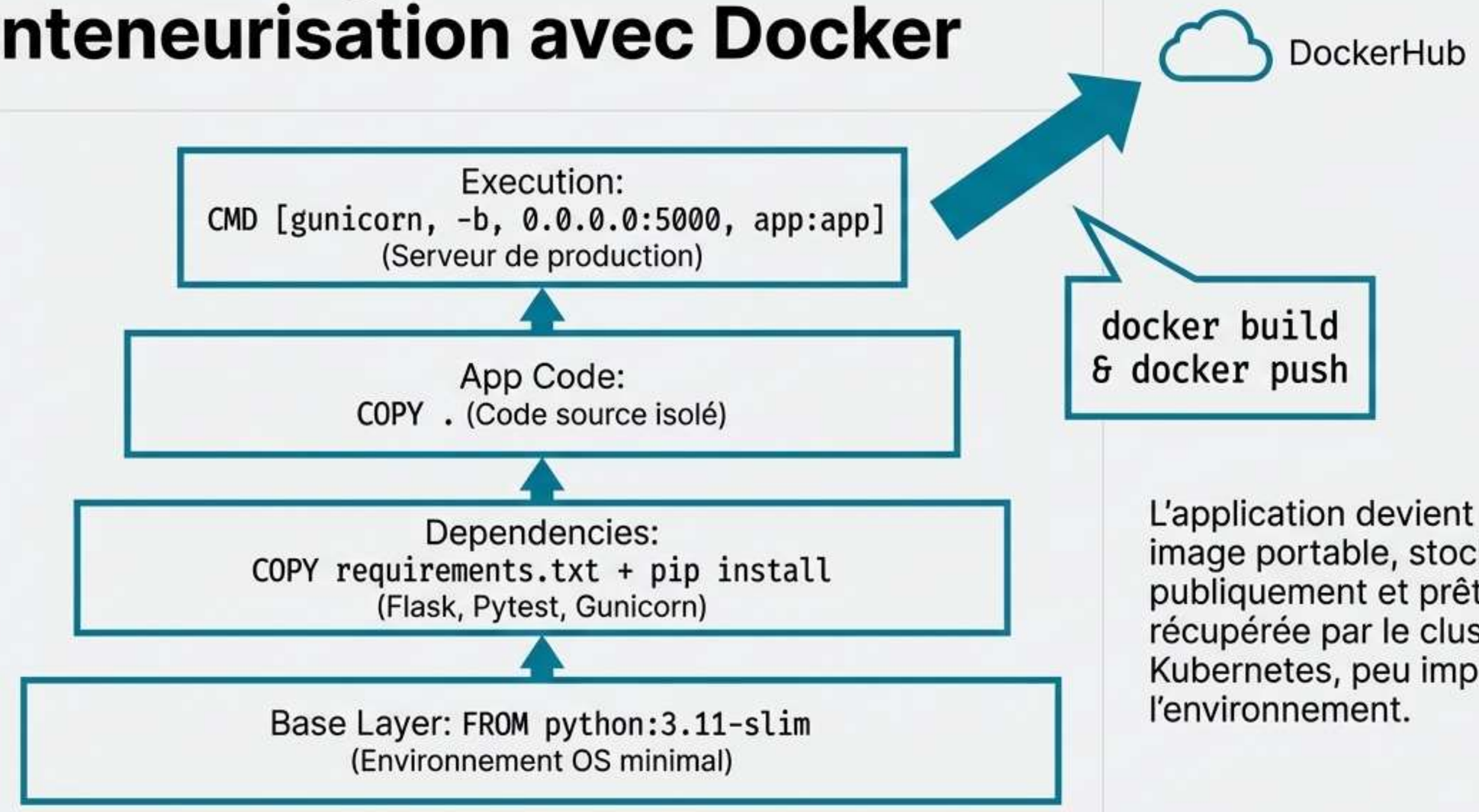
Utilisée par Kubernetes pour les Health Checks.

Tests Locaux (pytest)

✅	[v] `test_home()` -> Validation Code HTTP 200
✅	[v] `test_health()` -> Validation Code HTTP 200

Une application monolithique légère en Python 3.11, conçue spécifiquement pour valider la communication entre tous les composants de l'architecture.

L'isolation logicielle : Conteneurisation avec Docker

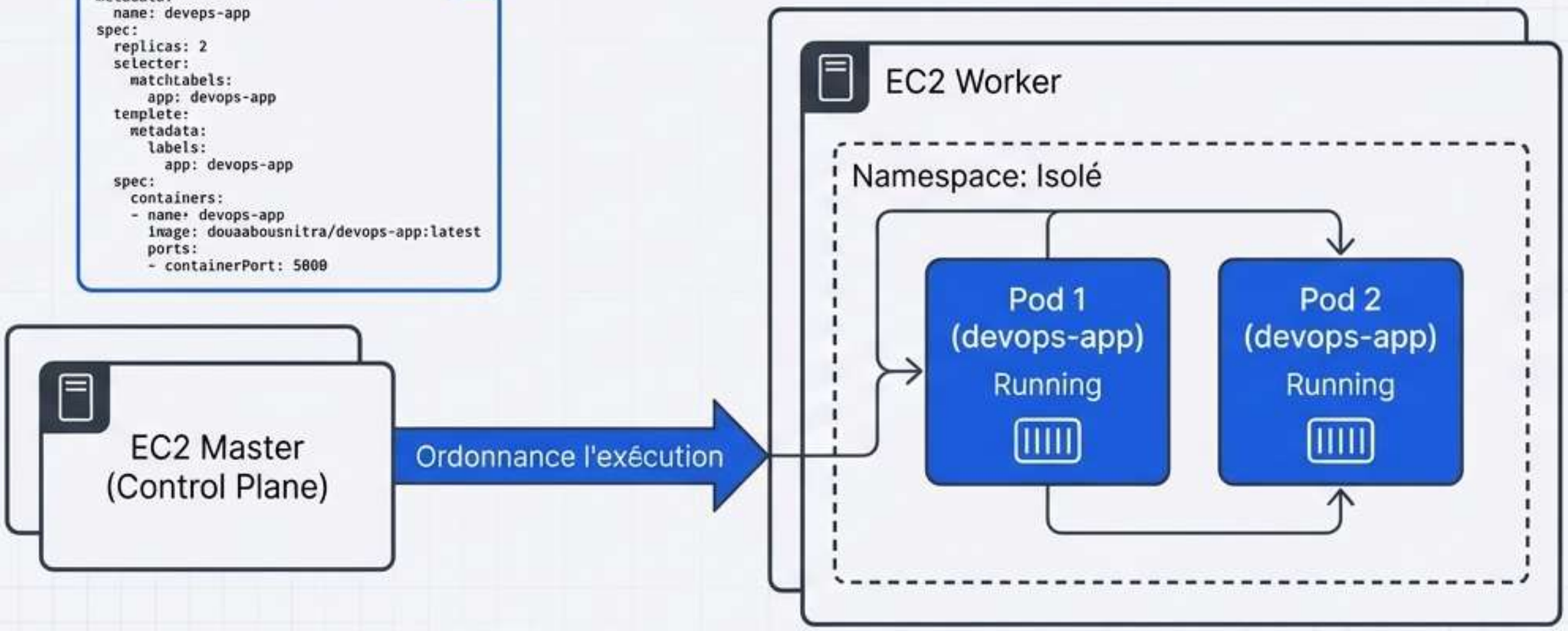


L'application devient une image portable, stockée publiquement et prête à être récupérée par le cluster Kubernetes, peu importe l'environnement.

Le cerveau de l'orchestration : Kubernetes (Control Plane)

Deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deveps-app
spec:
  replicas: 2
  selecter:
    matchtabels:
      app: devops-app
  templete:
    metadata:
      labels:
        app: devops-app
  spec:
    containers:
    - name: devops-app
      image: douaabousnitra/devops-app:latest
      ports:
      - containerPort: 5000
```



Haute Disponibilité : Le fichier Deployment spécifie 'replicas: 2'. Si un Pod échoue, le nœud Master le détecte via la route /health et recrée instantanément un nouveau conteneur pour garantir la continuité de service.

Le routage du trafic : L'exposition via NodePort

Utilisateur (Internet)
Envoie une requête HTTP

AWS EC2 Public IP
Point d'entrée cloud

Port 30080 (NodePort)
Service Kubernetes intercepte le trafic

Répartition de charge (Service)
Redirige vers le pod disponible

Port 5000 (TargetPort)
Conteneur Docker (Gunicorn/Flask)

Le fichier `Service.yaml` agit comme un pont sécurisé et un répartiteur de charge entre l'internet public et les conteneurs isolés.

Le moteur d'automatisation : **GitHub Actions**

Événement : `git` push **sur la branche** `[main]`

→

Étape 1 : `actions/checkout@v3`

Récupération du code

→

Étape 2 : `pytest`

Validation automatique.

Branche conditionnelle : Si échec -> Arrêt immédiat

→

Étape 3 : `docker build & push`

Génération et stockage de l'image

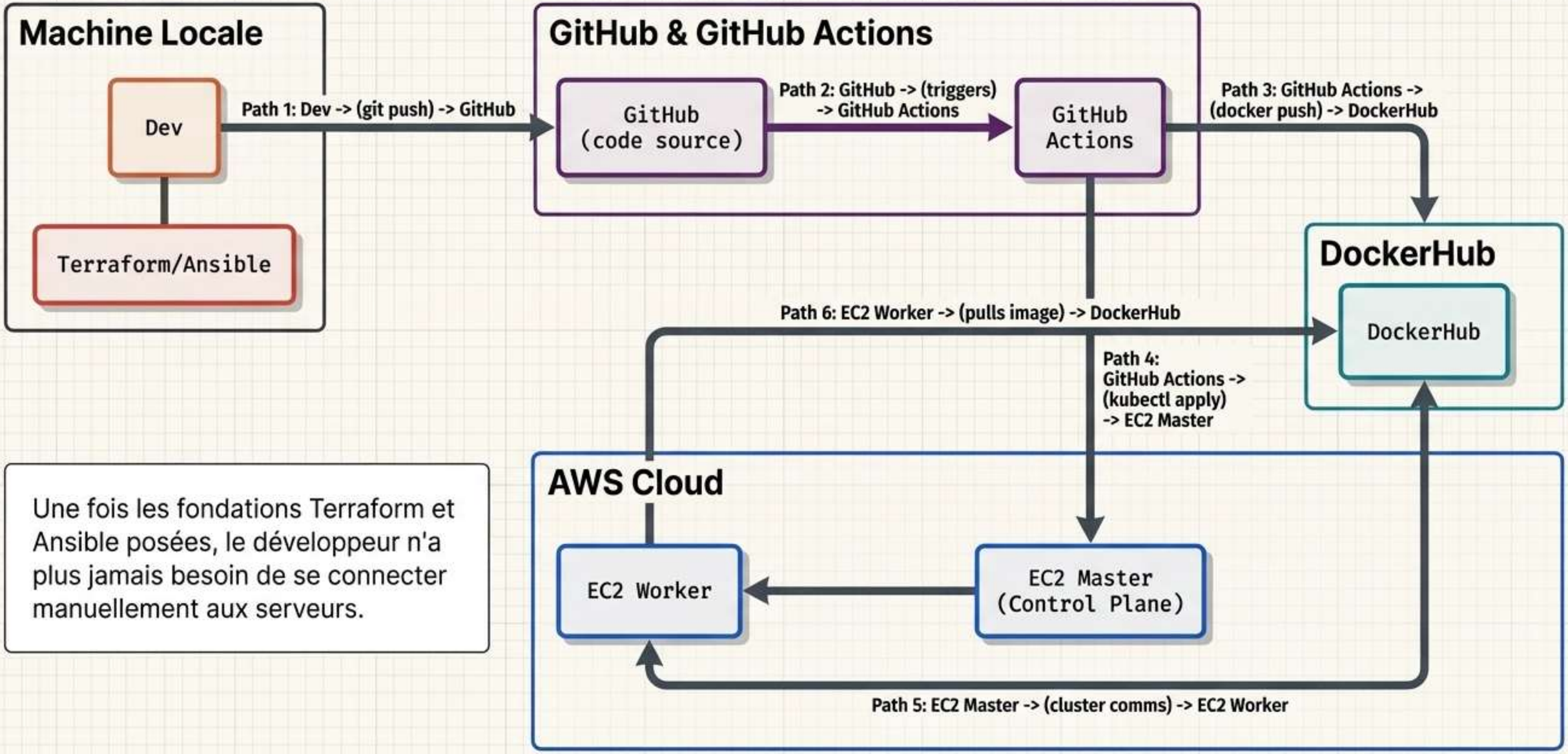
→

Étape 4 : `kubectl apply`

Mise à jour de l'infrastructure de production via YAML

L'intervention humaine s'arrête au git push. **Le pipeline CI/CD garantit que chaque modification est testée, construite et déployée en production en quelques minutes, de manière totalement fiable.**

La synthèse : Le voyage automatisé du code



Une fois les fondations Terraform et Ansible posées, le développeur n'a plus jamais besoin de se connecter manuellement aux serveurs.

Bilan : Une infrastructure moderne et résiliente

Vitesse (CI/CD)

L'intégration et le déploiement continus via `GitHub Actions` réduisent le temps de mise en production de plusieurs heures à quelques minutes.

Fiabilité (Kubernetes)

Les `Health Checks` et l'orchestration garantissent que l'application s'auto-répare et reste disponible sans intervention humaine.

Reproductibilité (Terraform & Ansible)

L'**Infrastructure as Code** permet de détruire et de recréer l'environnement AWS complet à l'identique, à la demande.

Portabilité (Docker)

L'isolation par **conteneurs** assure que l'application s'exécutera de la même manière en local que sur les serveurs de production.

Le DevOps n'est pas qu'un ensemble d'outils, c'est une ingénierie de l'automatisation.